

AI-SA-IN-TH-000002

Module 8.1 · LLM Evaluation Fundamentals · AI Safety: Evaluation, Guardrails & Responsible Deployment

LLM-as-a-Judge

Scalable evaluation through model-based scoring — paradigm, biases, and mitigations

9 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

Theory

Domain

Learning Objectives

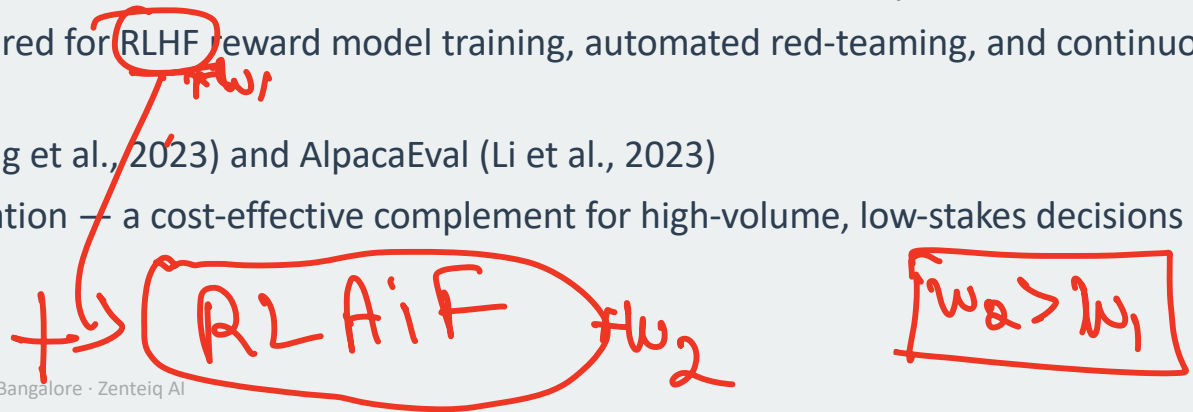
- 1 Explain the LLM-as-judge paradigm and its cost-scaling advantage over human evaluation
- 2 Distinguish pairwise, single-response, and reference-guided evaluation modes
- 3 Identify the three systematic biases (position, verbosity, self-preference) with worked examples
- 4 Apply three mitigation strategies to reduce judge bias in evaluation pipelines
- 5 Compare MT-Bench and AlpacaEval as LLM-as-judge evaluation frameworks
- 6 Select an appropriate judge model for a given cost-quality tradeoff

The Paradigm — LLM Evaluates LLM

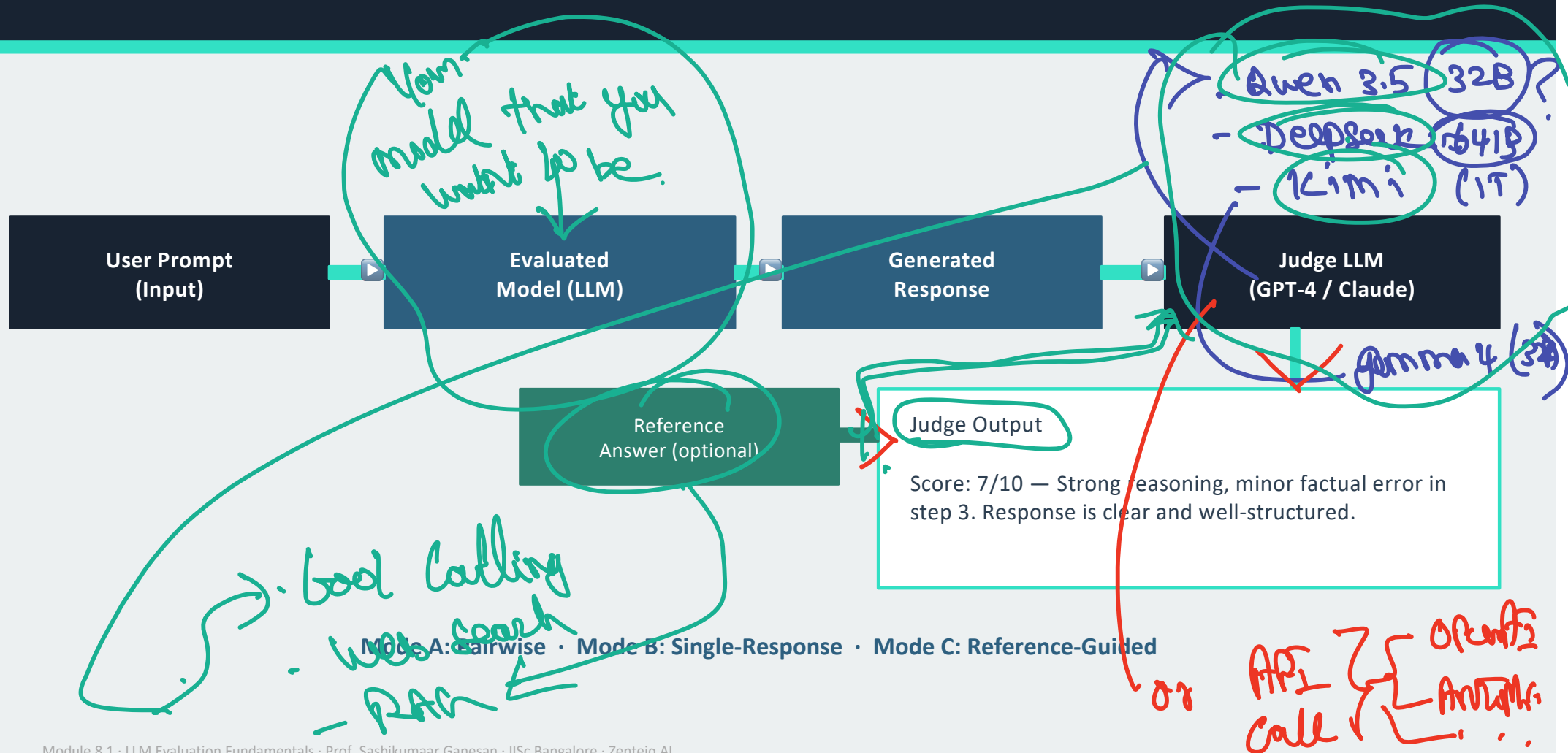
LLM-as-a-Judge

A paradigm in which a capable language model (the 'judge') is prompted to evaluate the quality of outputs generated by another language model (the 'evaluated model'). The judge assigns scores, makes pairwise comparisons, or provides detailed feedback — replacing or augmenting expensive human annotators.

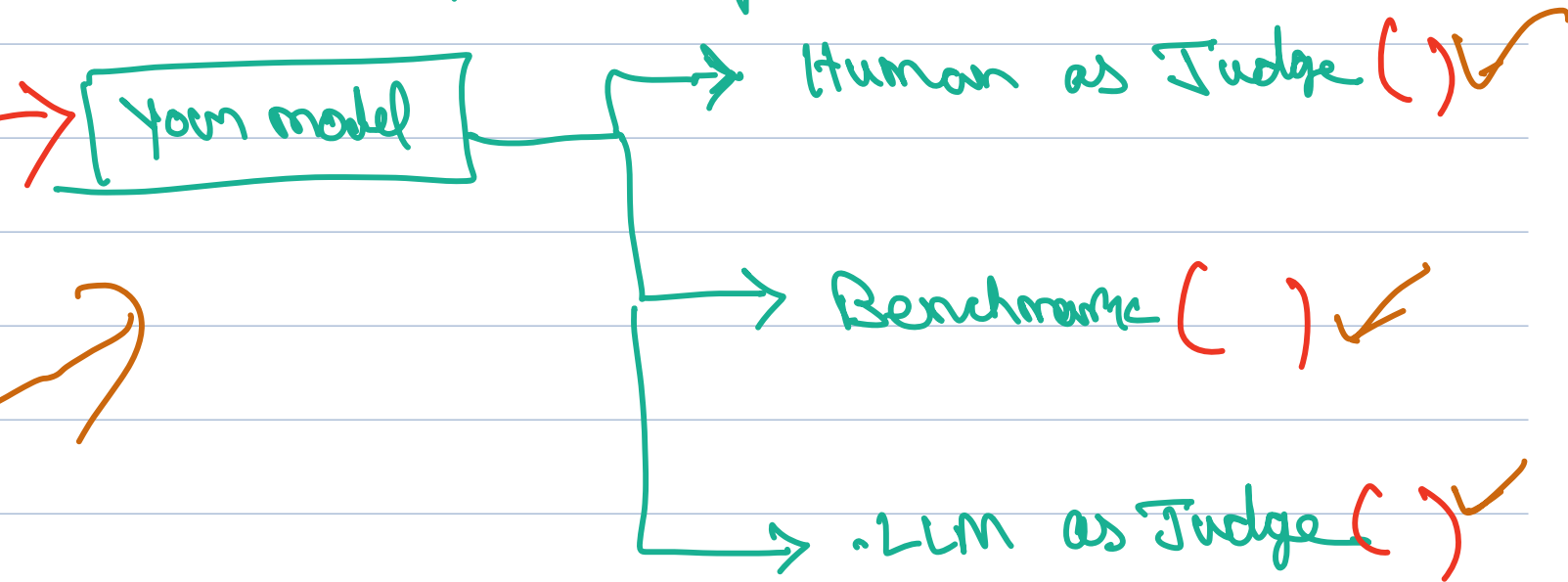
- Proposed as a scalable alternative to human evaluation: one API call costs ~\$0.001 vs ~\$2–5 per human annotation
- Enables evaluation at the scale required for RLHF reward model training, automated red-teaming, and continuous CI/CD
- First popularised by MT-Bench (Zheng et al., 2023) and AlpacaEval (Li et al., 2023)
- Not a replacement for human evaluation — a cost-effective complement for high-volume, low-stakes decisions



LLM-as-Judge Evaluation Pipeline



Step 1: Build Eval Platform

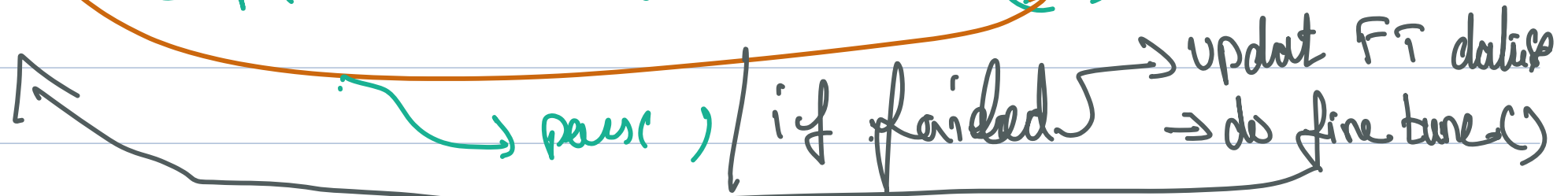


Step 2: I identify agent (any open weight model)

(m1) - GPT-3.5

Step 3: Based on the initial evaluation gaps,
Apply Fine-tuning()

Step 4: Call Evaluation()



Step 5: Deployment (Register as an Agent)

Why LLM-as-Judge Scales — Cost Economics

Cost comparison per 1,000 evaluation decisions (GPT-4o judge vs crowdworked human)

\$1–3

LLM-as-Judge Cost
per 1,000 evaluations
(GPT-4o API, 2024)

\$2,000+

Human Annotation Cost
per 1,000 evaluations
(crowdwork, trained annotators)

~10 s

Judge Latency
per evaluation
(API call, async)

~2 days

Human Turnaround
per batch
(queue + review time)

Evaluation Mode A — Pairwise Comparison

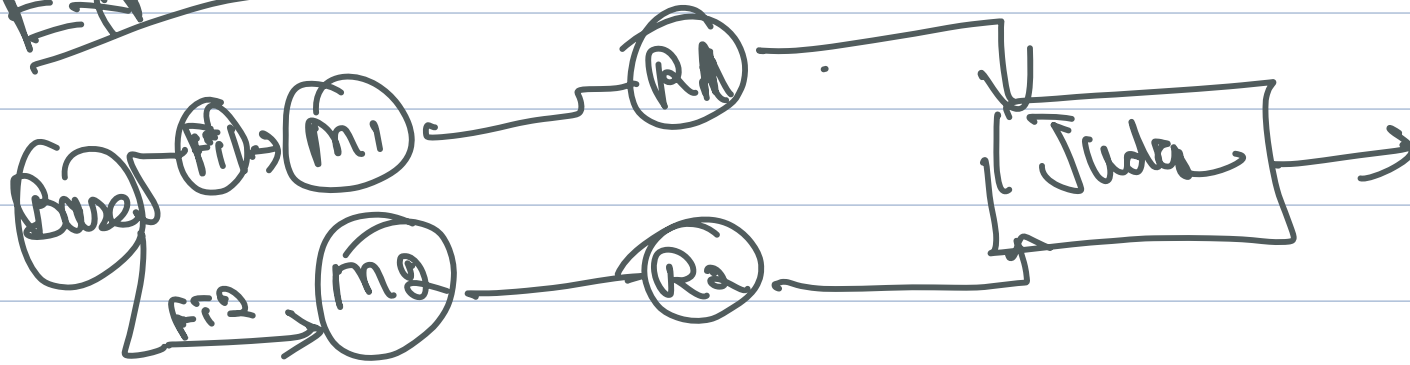
Pairwise Comparison

→ model outputs

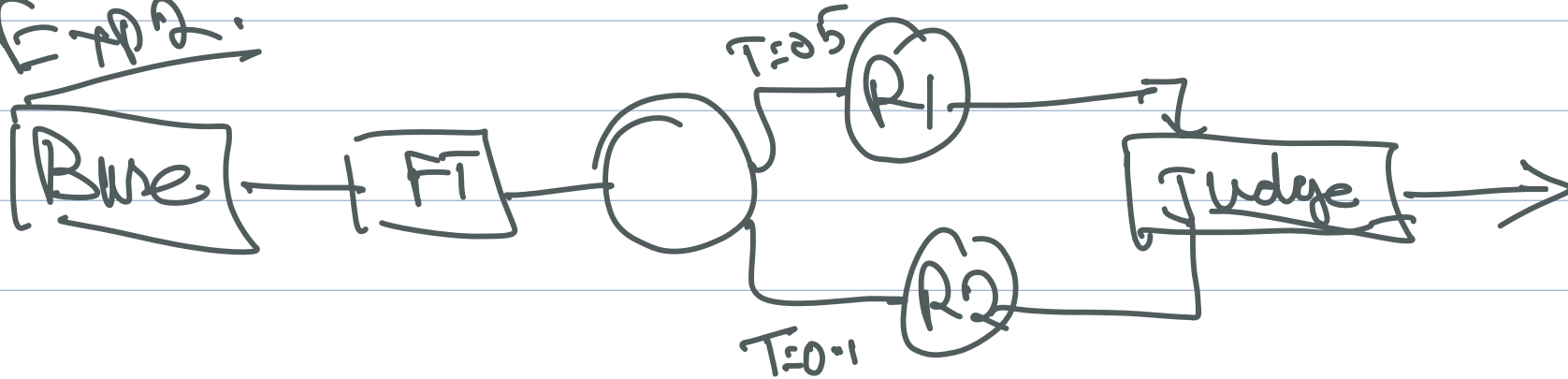
The judge is presented with a prompt and two candidate responses (Response A and Response B) and must declare which response is better, or that they are tied. This mirrors the approach used in RLHF preference data collection and in Chatbot Arena.

- Directly measures relative quality — doesn't require defining an absolute quality scale
- Aggregated into win-rate statistics: Win% / Tie% / Lose% across many prompt-response pairs
- More reliable than absolute scoring because it avoids scale anchoring issues
- Higher cost: requires 2 responses per prompt, and $O(n^2)$ comparisons for n models
- Standard in MT-Bench, AlpacaEval, Chatbot Arena, and most RLHF preference data pipelines

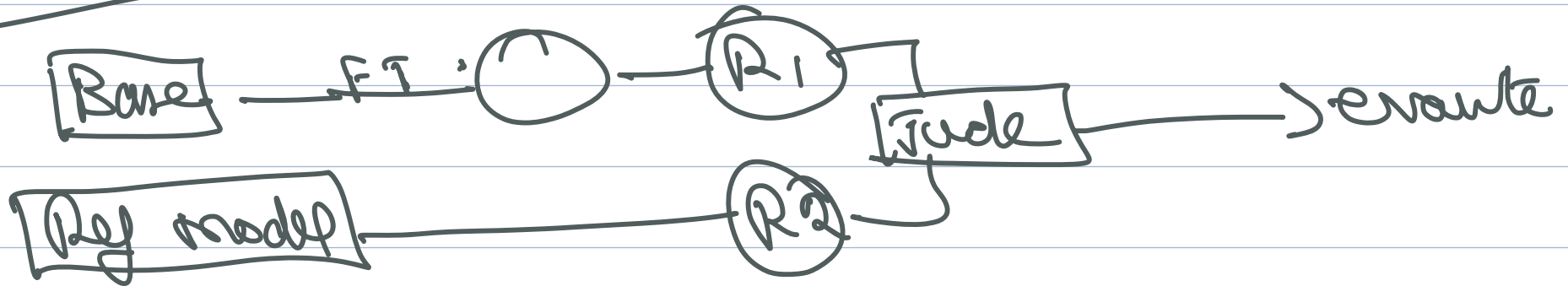
Exp 1:



Exp 2:



Exp 3:



Pairwise Comparison — Prompt Template

Prompt Template

```
SYSTEM: You are an expert evaluator. Your task is to assess two AI assistant responses and determine which is better. Be objective and analytical.
```

```
USER:
```

```
[Question]
```

```
{user_question}
```

```
[Response A]
```

```
{response_a}
```

```
[Response B]
```

```
{response_b}
```

```
Which response is better? Answer EXACTLY with one of:
```

```
Response A | Response B | Tie
```

```
Then provide a 2-sentence justification.
```

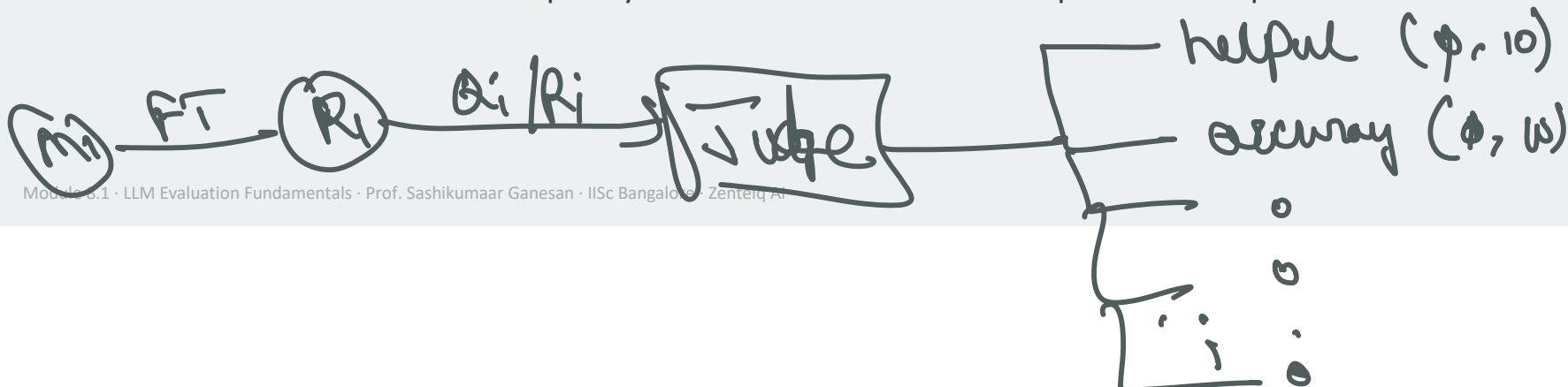
Parse with regex: `r'(Response A|Response B|Tie)'` from first line. Justification in subsequent lines.

Evaluation Mode B — Single-Response Scoring

Single-Response Scoring (Absolute Scoring)

The judge is presented with a single response and asked to score it on a defined Likert scale (typically 1–10) across one or more quality dimensions. No reference answer is required, enabling evaluation of tasks where ground truth is unavailable.

- Directly produces a scalar score — convenient for continuous monitoring and regression testing
- Requires careful rubric design: without a clear anchor, scores are inconsistent across sessions
- Common dimensions: helpfulness (1–10), accuracy (1–10), safety (1–10), coherence (1–10)
- Faster and cheaper than pairwise — only one response per prompt needed
- Limitation: absolute scores are poorly calibrated without reference points — use pairwise for final model selection



Single-Response Scoring — Prompt Template

Prompt Template

```
SYSTEM: You are an expert AI evaluator. Score the response on a scale of 1-10  
where 1 = completely unhelpful/wrong and 10 = perfect response.
```

```
Scoring rubric:
```

```
1-3: Major errors, off-topic, or harmful  
4-6: Partially correct but with significant gaps  
7-8: Correct and helpful with minor issues  
9-10: Excellent; accurate, clear, and complete
```

```
USER:
```

```
Question: {user_question}
```

```
Response: {model_response}
```

```
Provide score (integer 1-10) on the first line.
```

```
Provide 2-sentence justification on the second line.
```

Always include a scoring rubric. Without anchors, GPT-4 tends to cluster scores between 6–8.

Evaluation Mode C — Reference-Guided Scoring

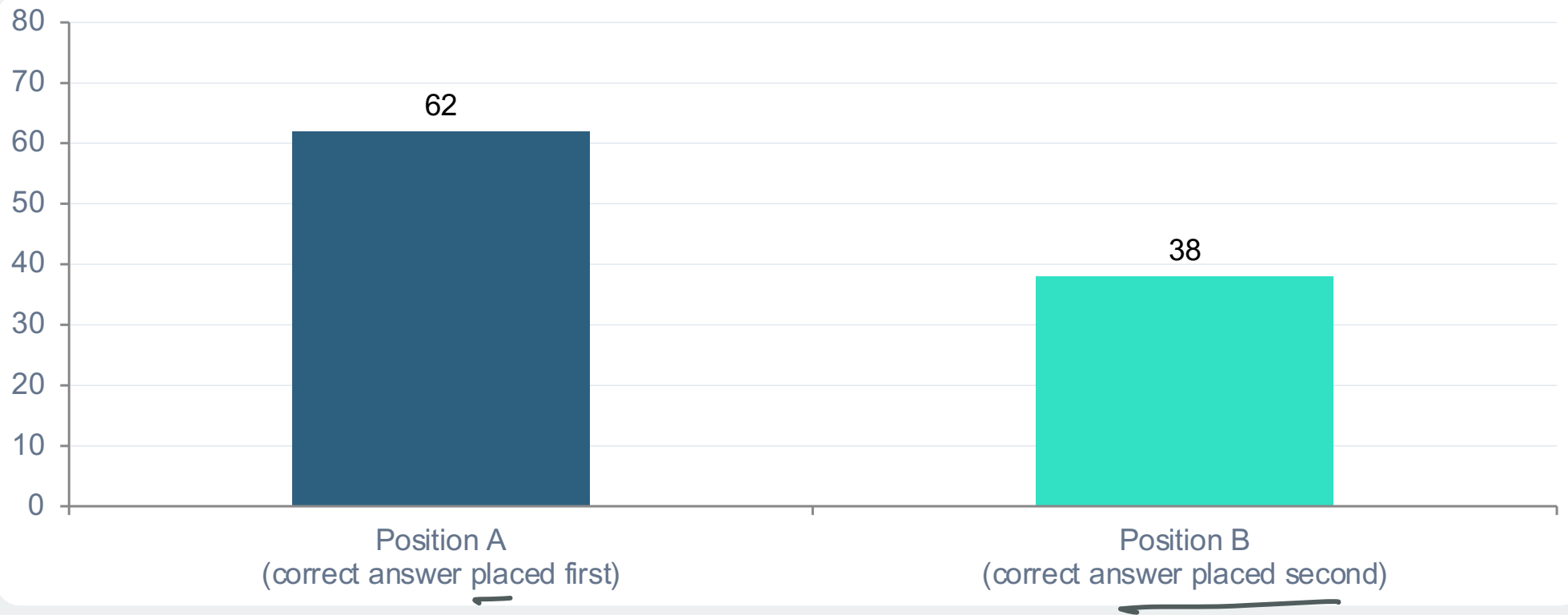
Reference-Guided Scoring

- Judge compares the candidate response against a gold-standard reference answer
- More reliable than absolute scoring: the reference provides a calibration anchor
- Enables factuality checking: judge can identify facts in the candidate that contradict the reference
- Required for tasks with clear right answers: trivia QA, medical guidelines, legal statutes
- Reference must be expert-curated — poor references produce worse judge performance than no reference

When to Use Reference-Guided

- Factual Q&A: reference = authoritative answer from verified source
- Summarisation: reference = human-written gold summary
- Code generation: reference = correct implementation with test cases
- Translation: reference = professional human translation
- Medical / legal: reference = verified expert response; mandatory for safety-critical eval

Position Bias — GPT-4 Prefers Response A by ~60%



Source: Zheng et al. (2023) MT-Bench analysis. Even GPT-4 shows ~60% preference for whichever response appears first. This bias is systematic and persists across different judge models unless mitigated.

Position Bias — Worked Example

Same two responses, swapped order — different judge verdict

Trial 1: Response A = Model X, Response B = Model Y

Prompt: "Explain gradient descent in 2 sentences."

Response A (Model X): "Gradient descent iteratively updates model parameters by moving in the direction of steepest descent of the loss function. The update magnitude is controlled by the learning rate."

Response B (Model Y): "Gradient descent minimises a loss function by computing partial derivatives and adjusting weights proportionally. Each step reduces the loss by a small amount determined by the learning rate."

Judge verdict: Response A wins (62% probability)

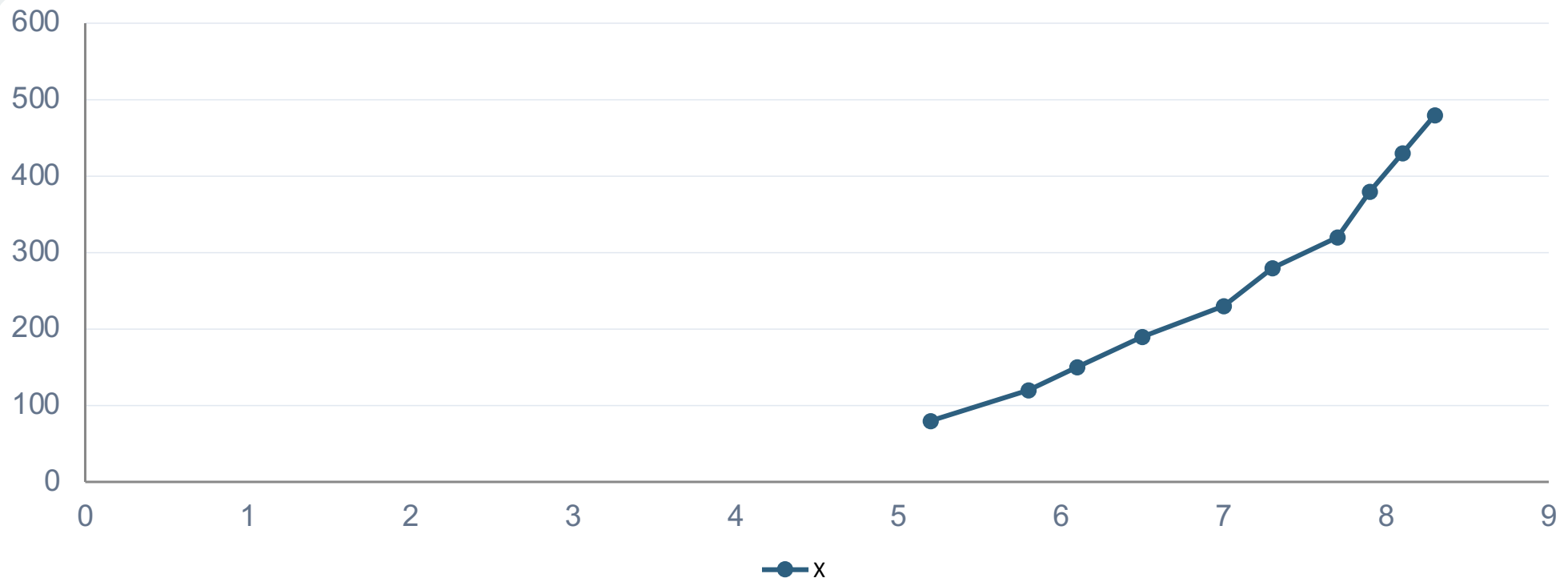
Trial 2: Swapped — Response A = Model Y, Response B = Model X

Identical responses, reversed order.

Judge verdict: Response A wins again (58% probability)

Conclusion: Judge assigns higher probability of winning to whichever response appears in position A, regardless of quality. This inflates win rates for models whose responses happen to be assigned to position A.

Verbosity Bias — Longer Responses Score Higher



Source: Dubois et al. (2023) AlpacaFarm analysis. Correlation $r \approx 0.61$ between response length and judge score, controlling for task difficulty. Longer is not necessarily better — but judges systematically reward verbosity.

Self-Preference Bias

Self-Preference Bias (aka Self-Enhancement Bias)

When the same model family is used as both the evaluated model and the judge, the judge systematically assigns higher scores to responses generated by itself or by models trained with similar techniques, even when objective quality is identical.

- Mechanism: models trained on similar data and with similar RLHF preferences recognise their own style and reward it
- Measured: GPT-4 judges assign GPT-4 responses a win rate ~10–15% higher than human judges would assign
- Documented in: PandaLM (Wang et al., 2023), Self-Rewarding LLMs (Yuan et al., 2024)
- Risk: using Claude-3.5-Sonnet to evaluate models fine-tuned from Claude base models biases scores upward
- Mitigation: use a different model family as judge, or use multiple judges from different families

Three Systematic Biases — Summary

Bias	Description	Magnitude	Detection	Mitigation
Position Bias	Prefers response in position A	~60% vs 50% expected	Swap A/B order, compare verdicts	Randomise order; average both trials
Verbosity Bias	Scores longer responses higher	$r \approx 0.61$ length-score corr.	Correlate score with word count	Add length normalisation; penalise padding
Self-Preference	Favours own model family style	+10–15% win-rate inflation	Compare to human judgment on same pairs	Use different model family as judge

Mitigation 1 — Randomise Response Order

The simplest and most effective mitigation for position bias:

- For each (prompt, response_A, response_B) triple, evaluate twice: once with original order (A, B) and once with reversed order (B, A)
- Average the two verdicts: if both trials agree, use that verdict; if they disagree, call it a tie
- Reduces position bias from ~60%/40% split to ~52%/48% — near-chance — at the cost of 2× API calls
- Implementation: maintain a random seed per evaluation batch; ensure deterministic reversal pairing for reproducibility
- Cost: doubles evaluation cost but dramatically improves reliability — always worth it for final model selection decisions

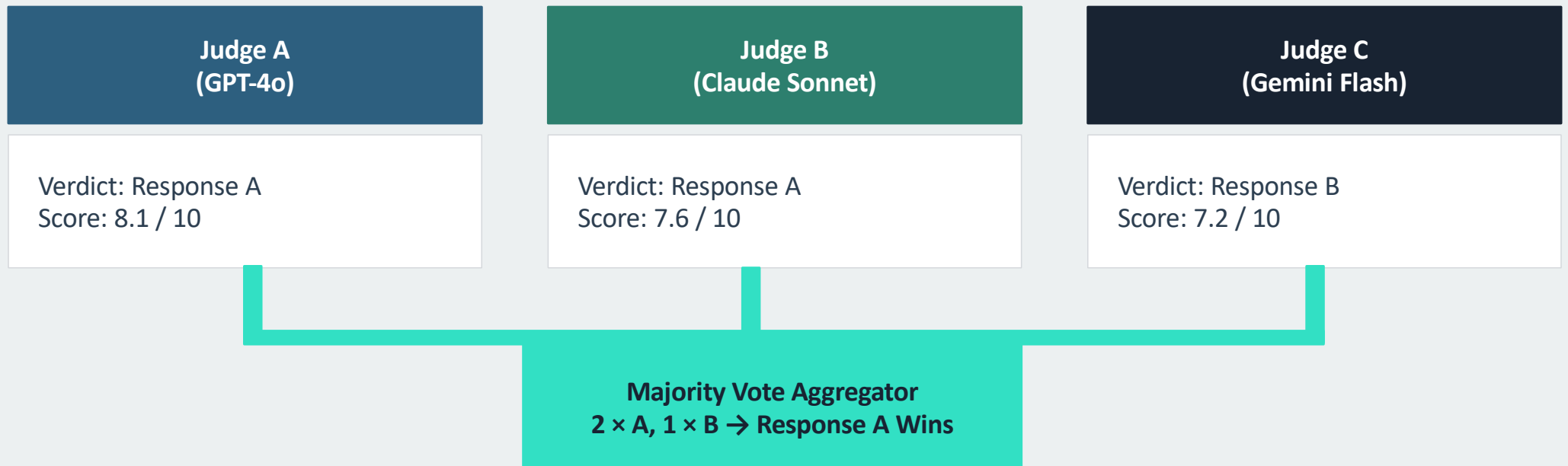
Mitigation 2 — Human Calibration & Agreement Score

$$\kappa_{\text{agreement}} = (P_{\text{observed}} - P_{\text{expected}}) / (1 - P_{\text{expected}})$$

1. Sample 100–200 evaluation pairs and obtain both LLM judge verdicts and human annotator verdicts
2. Compute P_{observed} : fraction of pairs where judge and human agree
3. Compute P_{expected} : expected agreement by chance (0.33 for 3 options: A wins, B wins, Tie)
4. Compute Cohen's κ : $\kappa = (P_{\text{obs}} - P_{\text{exp}}) / (1 - P_{\text{exp}})$. Target $\kappa > 0.6$ for reliable judge
5. If $\kappa < 0.4$: recalibrate judge prompt, switch judge model, or add human review for low-confidence decisions

Note: Calibration is not a one-time activity — rerun monthly as evaluated models evolve

Mitigation 3 — Multi-Judge Ensemble



Ensemble reduces single-judge bias by ~40%. Use odd number of judges (3 or 5) to avoid ties.

Cost-quality tradeoff: 3-judge ensemble at \$3 vs \$2,000 for human annotation — 99.9% cost saving with ~85% agreement accuracy.

MT-Bench — LLM-as-Judge Framework

MT-Bench (Zheng et al., 2023 — LMSYS)

MT-Bench is a multi-turn benchmark of 80 high-quality two-turn conversation problems across 8 categories (writing, roleplay, reasoning, math, coding, extraction, STEM, humanities). GPT-4 serves as the judge, scoring responses on a 1–10 scale. It was the first widely-adopted LLM-as-judge framework.

- 80 questions × 2 turns = 160 evaluations per model; covers both single-turn quality and multi-turn instruction-following
- 8 categories test diverse capabilities: not just knowledge, but creativity, multi-step reasoning, and format adherence
- GPT-4 judge with temperature=0 for reproducibility; Pearson correlation with human: 0.89
- First shown that GPT-4 as a judge achieves human-agreement on pairwise comparisons ~80% of the time
- Limitation: 80 questions is small; modern models may have memorised MT-Bench prompts; use alongside AlpacaEval and Chatbot Arena

AlpacaEval vs MT-Bench — Framework Comparison

MT-Bench

- 80 two-turn conversation problems
- 8 capability categories (writing, math, code, etc.)
- Absolute scoring (1–10 per response)
- GPT-4 judge; high reproducibility
- Pearson $r = 0.89$ with human judgment
- Best for: assessing instruction-following quality across diverse task types
- Limitation: small prompt set; saturation risk

AlpacaEval 2.0

- 805 diverse instructions from various sources
- Pairwise: model response vs GPT-4-turbo reference
- Reports win rate vs reference model
- LC (length-controlled) win rate addresses verbosity bias
- Pearson $r = 0.97$ with Chatbot Arena ELO
- Best for: quick, cheap proxy for Chatbot Arena ranking
- Current: AlpacaEval 2.0-LC is the most trusted automated alternative

Judge Model Selection — Cost vs Quality Tradeoff

Judge Model	Agreement w/ Human	Cost / 1K evals	Latency	Best For
GPT-4o	~82%	\$5–8	3–8s	Final model selection, high-stakes
Claude-3.5-Sonnet	~81%	\$4–7	3–6s	General-purpose; good safety eval
Gemini-1.5-Pro	~80%	\$3–6	4–8s	Ensemble diversity; multimodal
GPT-4o-mini	~74%	\$0.5–1	1–3s	Rapid iteration, dev-time eval
LLaMA-3-70B-Instruct	~71%	\$0.1	2–4s	Self-hosted, privacy-sensitive
Prometheus-2-8x7B	~75%	\$0.08	2–5s	Fine-tuned for evaluation; OSS

Key Takeaways

AI-SA-IN-TH-000002

- 1 LLM-as-judge scales to millions of evaluations at \$1–3 per 1,000 decisions — a 99%+ cost reduction vs human annotation
- 2 Three evaluation modes serve different needs: pairwise for model comparison, single-response for monitoring, reference-guided for factual tasks
- 3 Position bias (~60% preference for position A) and verbosity bias ($r \approx 0.61$ with length) are the most dangerous systematic errors
- 4 Mitigate position bias by always evaluating both A/B orderings and averaging; mitigate verbosity bias with length-controlled metrics
- 5 Self-preference bias inflates win rates by 10–15% when judge and evaluated model share training lineage — always use a different model family as judge
- 6 MT-Bench and AlpacaEval 2.0-LC are the two most-trusted LLM-as-judge frameworks; use in combination with Chatbot Arena for production model decisions